Mini SIEM

Notes on building a detection platform from scratch

github.com/sodik-tursunboev/SIEM

I kept running into the same problem doing security labs and courses: you learn to point Splunk at some sample logs, write a search, and that's supposed to count as understanding a SIEM. It doesn't. You never see what's actually happening underneath - how an event gets parsed, what decides it's suspicious, why a correlation engine groups five alerts into one incident instead of leaving them scattered. So I built one myself. Not a toy - a real Flask application that ingests actual Windows and Linux telemetry, runs it through detection logic I wrote and tuned by hand, and responds to it roughly the way a production SOC tool would.

This document is my own writeup of how it works and why I built each piece the way I did, including the parts that didn't work the first time. It's meant to be read by someone deciding whether to hire me, so I've tried to be honest about what's solid and what's still rough rather than polish over the gaps.

What it actually does, day to day

Most of the time it's quiet. Logs come in from wherever they're pointed - the local machine, a Linux box over syslog, a Windows VM running the forwarder agent - and nothing happens, because nothing suspicious is actually going on. That's correct behavior, and it took me longer than I'd like to admit to stop treating a quiet dashboard as something broken.

When something does trip a rule, the alert gets tagged with the MITRE technique it matches. If it's not alone - if the same user or IP has set off two or three other distinct rules in the last hour - the correlation engine notices and groups them into one incident instead of leaving an analyst to piece it together by hand. That correlation piece is the part of this project I'm most proud of. It isn't a hard algorithm. But it's the thing that made this stop feeling like a list of alerts and start feeling like something that understands what an attack actually looks like.

The detection rules - a few worth talking about specifically

There are 35 rules right now, covering everything from basic brute force to process injection and DNS tunneling. The full list is in the appendix at the end - here I want to talk about a few that were worth building carefully, and one I had to go back and fix.

Kerberoasting

This one only makes sense if you understand what it's not looking for. A single Kerberos service ticket request is completely normal - it happens constantly on any Windows domain. What isn't normal is a burst of them, using weak RC4 encryption specifically, against multiple different service accounts, from one user, in a short window. That's what tools like Rubeus and Impacket's GetUserSPNs actually do when they're harvesting every roastable account they can find. The rule waits for three of those RC4 requests inside ten minutes before it fires. One is noise. Three is a pattern.

The LSASS rule I had to rebuild

My first version of LSASS credential-dumping detection just checked whether the word "lsass" showed up in a process's command line or message text. It worked well enough as a first pass, but it bothered me - it was a keyword match standing in for something that should be a real security check. Sysmon's ProcessAccess event actually reports the exact Windows access rights a process requested when it opened a handle to another process. Real credential-dumping tools ask for very specific rights combinations against lsass.exe - things like 0x1010 or 0x1438 - that ordinary software essentially never requests. So I rebuilt the rule to check the actual access mask instead of just hoping the word "lsass" showed up somewhere nearby. It's a more honest detection now.

Office spawning a shell

Word or Excel launching cmd.exe or PowerShell is about as close to a guaranteed red flag as this field gets - it's the signature of a malicious macro firing the moment someone opens an attachment. The rule checks that the parent process is genuinely an Office application (not just that the word "winword" shows up somewhere in the log line) and that the child is a shell or scripting engine. First version of this regex broke on real Windows paths because "C:\Program Files\..." has a space in it, which I hadn't accounted for. Found it by testing against a realistic path instead of a clean fake one, which is a mistake I try not to repeat now.

Attack chain correlation

The rule is simple to state: if the same user or IP sets off three or more distinct rule types within an hour, that's treated as one coordinated incident instead of independent noise. Distinct rule types matter more than raw count on purpose - five failed logons is still just one brute-force attempt, but a brute force followed by a privilege escalation followed by a credential dump is a genuinely different, much more serious situation, even though it's also technically three alerts either way.

I want to be upfront that this is a heuristic, not a certainty. A busy admin account doing several unusual-but-legitimate things inside an hour could, in theory, false-positive into a chain. That's exactly why a chain queues for an analyst to look at rather than automatically opening a case or taking any action on its own - the tool notices the pattern, a person decides what it means.

When an analyst does confirm it's real, one click turns the whole chain into a case with every alert in it attached as evidence automatically, and the case priority gets set from the worst severity alert in the chain rather than just the most recent one.

Response, and why nothing happens automatically

The SOAR side queues actions - blocking an IP, disabling an account - whenever an alert matches a playbook. It does not execute anything on its own. Every single action sits pending until an analyst clicks approve, and even then, hard guardrails apply no matter what: it will refuse to block a private IP range or disable a protected account, full stop, regardless of who approved it or how the alert looked. I built it this way on purpose. An automated response system that can take down your own infrastructure is a worse outcome than the incident it was supposed to be responding to.

Getting Sysmon and the remote forwarder actually working

This is worth writing about honestly because it wasn't a clean process, and I think the mess is more informative than a tidy feature description would be.

The first real problem: Windows doesn't log the actual command line on a process-creation event by default. The event fires, but the field that would tell you what was actually run is empty unless a specific audit policy is turned on. I only found this by looking at real Event Viewer output during testing and noticing the command line field was just missing - not a bug in my code, a Windows default that has to be explicitly changed.

Sysmon solves that - it logs full command lines without any extra configuration - but my collector only read the Security and System channels at first. Sysmon has its own completely separate event numbering scheme, and I hadn't wired it into the pipeline at all, so the richer telemetry was sitting right there and I wasn't reading it.

Once I added Sysmon collection, I hit a second, sneakier problem: the remote forwarder agent for VMs sent Sysmon's process-creation events under their raw Sysmon event ID, but the detection rules that actually catch Atomic Red Team-style attacks are written to check for Windows' native process-creation ID specifically. So the events arrived, the command line was right there in the message, and the rule still never fired - because the ID check failed before the message content was ever read. I proved this by sending the exact same payload twice, once under each ID, and watching the alert count go from zero to real. The fix was remapping Sysmon's process-create events to look like the native ID before they reach the rule engine.

And then, separately, a genuinely dumb one: the test VM and the host machine had the same hostname, because the VM had never been renamed after cloning. Every log from both machines looked identical in the dashboard, which made it look like data wasn't arriving from the VM at all when really it was arriving fine, just indistinguishable from the host's own activity. Renamed the VM, problem gone.

None of these were exotic bugs. They were the kind of thing you only find by actually running an attack and watching what happens instead of assuming the pipeline works because the code compiles.

A real security bug in the tool itself

During a self-audit I found that Sigma rule titles - which come from user-uploaded YAML content - were being inserted into several pages completely unescaped. I didn't just read the code and assume it was fine. I uploaded a Sigma rule with an actual script payload in its title and watched it execute in the browser, which is the only way to actually know a stored XSS is real rather than theoretical. After the fix I ran the identical attack again and confirmed it now renders as plain, inert text.

The fix itself had a subtlety worth mentioning: simple HTML escaping isn't enough for values that get embedded inside an inline onclick handler, because the browser decodes the HTML attribute before the JavaScript engine ever parses it as code - so an escaped quote character can still reopen a string and break out. I had to write a second, separate escaping function specifically for that case.

While I was auditing, I also noticed the tool that detects brute-force attacks had zero protection against brute force on its own login page, which would have been an embarrassing thing for an interviewer to point out. Added a lockout, and - the part I actually like about this fix - routed failed login attempts through the exact same

detection pipeline as every other log source, so an attack on the SIEM's own front door now shows up as a real, MITRE-tagged alert instead of disappearing into a separate log nobody watches.

Why the interface looks the way it does now

The first version leaned hard into a dark-terminal, phosphor-green look, which is fun but reads more like a hacker aesthetic than something a SOC analyst would actually trust for eight hours a day. I rebuilt it around what real tools in this space - Splunk, Elastic Security, Sentinel - actually look like: calm dark neutrals almost everywhere, a left sidebar instead of a top bar because that's what scales to more than a dozen pages without feeling cluttered, and exactly one accent color reserved for critical and high severity so that when something is red, it actually means something.

Monospace font is used specifically for data - timestamps, IPs, hostnames, rule IDs - and a plain sans for everything else, which is a deliberate ratio change from the earlier version where almost everything was monospace. Most dashboards mix fonts loosely. A real console tool doesn't.

How I actually know it works

Every feature in this project got tested against a live, running instance before I considered it done - not unit tests in isolation, actual attack payloads sent through the real pipeline, checked against real output. The detection rules specifically were validated against real Atomic Red Team test runs on an actual Windows Server VM, which is where most of the debugging described above came from.

What isn't done: there's no automated test suite committed to the repository. Everything above was tested extensively during development, but that testing lived in my process, not in re-runnable code someone else could execute later. That's the next thing I'd build if I kept going, specifically so future changes don't rely on me remembering to retest everything by hand.

What this actually is, technically

Layer	
Backend	Python, Flask
Storage	SQLite - a real, known limitation, not a hidden one (see below)
Frontend	Vanilla JavaScript, a small hand-written SVG chart renderer - no framework, no external chart library
Detection	Custom rule engine + a Sigma YAML parser supporting the common AND/OR condition patterns
Ingestion	Local Windows Event Log + Sysmon, Linux syslog, a standalone PowerShell-based remote forwarder agent
Reporting	reportlab for PDF generation, optional AI summaries via Anthropic or Groq

Running it

```
git clone
cd mini-siem
pip install -r requirements.txt
python app.py demo # simulated data
python app.py windows # real Windows + Sysmon collection, run as Administrator
```

First login is admin / admin123. Change it immediately - the account lockout will otherwise apply to you too if you forget and mistype it a few times, which is a very findable way to learn the lockout logic works.

What I'd say if asked directly about the gaps

SQLite doesn't handle concurrent writes at real production log volume - fine for a lab, would need PostgreSQL or something search-optimized for a real deployment, and the detection logic doesn't care what's underneath it so that migration wouldn't mean rewriting rules. Everything runs over HTTP, not HTTPS, which is fine on an isolated VM network and not acceptable anywhere near the internet. The Sigma parser covers common condition patterns, not the entire specification. And the account lockout has the same tradeoff every lockout policy does - someone who knows a valid username could deliberately trigger it as a denial-of-service against that person, which is a known, accepted limitation rather than something I missed.

Appendix: full rule catalog

Rule	Severity	MITRE
Brute Force Detection	High	T1110
Password Spraying	High	T1110.003
Account Lockout	Medium	T1110
New Local Admin	Medium	T1098
Audit Log Cleared	Critical	T1070.001

Suspicious Process Execution	High	Varies
Suspicious PowerShell	High	T1059.001
New Scheduled Task	Medium	T1053.005
New Service Installed	Medium	T1543.003
USB Device Connected	Low	T1200
Kerberoasting	High	T1558.003
Suspicious Parent-Child Process	Critical	T1204.002
PowerShell Downgrade Attack	High	T1059.001
PowerShell Obfuscation Indicators	High	T1027
AMSI Bypass Attempt	Critical	T1562.001
Suspicious LSASS Access	Critical	T1003.001
Process Injection	Critical	T1055
DNS Tunneling	High	T1071.004
Discovery Command Burst	Medium	T1082
Living-off-the-Land Binary Abuse	High	Varies
Registry Run Key Persistence	High	T1547.001
Indicator Removal - Log/Artifact Deletion	Critical	T1070.004
Remote Service Execution	High	T1021.002
Shadow Copy / Backup Deletion	Critical	T1490
Unsecured Credential Search	Medium	T1552
Process Masquerading	Critical	T1036.005
SSH Brute Force	High	T1110
Linux Privilege Escalation	Medium	T1548
Suspicious Sudo Command	High	Varies
New Linux User Created	Medium	T1136.001
Linux User Added to Privileged Group	Critical	T1098
Unusual Login Hour	Medium	T1078
Login From New Source IP	Medium	T1078
SIEM Login Brute Force	Critical	T1110

IOC Watchlist Match	Critical	TA0043